example of how to use this header value to resolve the correct service to call based on the current request.

Listing 7.14 Resolving one from a number of registered services

```
public class CityModel : PageModel
{
    private readonly IEnumerable<IPriceService> _priceServices;
    public CityModel(IEnumerable<IPriceService> priceServices)     ❶
    {
        _priceServices = priceServices;
    }

    public void OnGet()
    {
        var locationCode = Request.Headers["CF-IPCountry"];        ❷
        var priceService = _priceServices.FirstOrDefault(s=> s.GetLocation()
        ➥ == locationCode);                                       ❸
        // do something with priceService
    }
}
```

❶ Inject a collection representing all registered implementations.

❷ Obtain the runtime data used to define the implementation applicable to this request.

❸ Query the collection for a service that matches the predicate passed in to the FirstOrDefault method.

There are two clear benefits to adopting this pattern. The first is that each `IPriceService` implementation is location specific, which reduces the amount of code they require and results in a simpler maintenance experience. The second is that if you want to cater to additional locations, you just need to create a new service and register it along with the others. It will automatically be resolved as part of the injected collection.

There is another way to register services that will result in the first registration to be resolved in the situation where multiple implementations are registered, instead of the last. That is to use the `TryAdd<LIFETIME>` method. If you repeat the registration of the `IPriceService` implementations using `TryAddScoped` (as shown